Exact File-by-File AI Prompts for Final Year Project Implementation

AI-Powered Adaptive Study Group Matching Platform
React + FastAPI + PostgreSQL + SQLAlchemy + JWT

Use this document as a direct copy-paste guide in VS Code or any AI coding assistant.

How to use this PDF

- Create the project folders first exactly as shown in the structure section.
- Go in the given order. Do not jump randomly between files.
- Copy one prompt at a time into your AI coding assistant in VS Code.
- After a file is generated, save it in the exact path mentioned in the heading.
- Run the project often so you catch errors early.
- Start with rule-based matching and dummy data. Keep ML as a later enhancement.

Recommended build order

1. 1. requirements.txt
2. 2. backend config.py
3. 3. backend database.py
4. 4. backend models
5. 5. backend schemas
6. 6. backend security.py
7. 7. auth router
8. 8. user and preference routers
9. 9. matching service
10. 10. match and group routers
11. 11. seed data script
12. 12. frontend auth and router files
13. 13. frontend pages and components
14. 14. dashboard polish and demo screenshots

Project folder structure

```
project-root/

├── backend/

|   ├── app/

|   |   ├── main.py
```

```
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── database.py
│   │   │   └── security.py
│   │   ├── models/
│   │   │   ├── user.py
│   │   │   ├── preference.py
│   │   │   ├── subject.py
│   │   │   ├── availability.py
│   │   │   ├── study_group.py
│   │   │   ├── match.py
│   │   │   └── message.py
│   │   ├── schemas/
│   │   │   ├── auth.py
│   │   │   ├── user.py
│   │   │   ├── preference.py
│   │   │   ├── match.py
│   │   │   └── study_group.py
│   │   ├── routes/
│   │   │   ├── auth.py
│   │   │   ├── users.py
│   │   │   ├── preferences.py
│   │   │   ├── matches.py
│   │   │   ├── groups.py
│   │   │   └── dashboard.py
│   │   ├── services/
│   │   │   ├── matching_service.py
│   │   │   ├── dummy_data_service.py
│   │   │   └── recommendation_service.py
│   │   └── seed/
│   │       └── seed_data.py
│   ├── requirements.txt
```

```
│   └── .env
├── frontend/
│   ├── src/
│   │   ├── main.jsx
│   │   ├── App.jsx
│   │   ├── api/axios.js
│   │   ├── components/
│   │   │   ├── Navbar.jsx
│   │   │   ├── Sidebar.jsx
│   │   │   ├── MatchCard.jsx
│   │   │   ├── GroupCard.jsx
│   │   │   └── ProtectedRoute.jsx
│   │   ├── pages/
│   │   │   ├── Login.jsx
│   │   │   ├── Register.jsx
│   │   │   ├── Dashboard.jsx
│   │   │   ├── Preferences.jsx
│   │   │   ├── Matches.jsx
│   │   │   ├── Groups.jsx
│   │   │   └── Profile.jsx
│   │   ├── context/AuthContext.jsx
│   │   └── router/AppRouter.jsx
├── README.md
└── .gitignore
```

Root Files

README.md

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a professional README.md for a final year project called "AI-Powered Adaptive Study Group Matching Platform".



Tech stack:

- React + Vite frontend

- FastAPI backend

- PostgreSQL database

- SQLAlchemy ORM

- JWT authentication



Include:

- project overview

- objectives

- main features

- tech stack

- folder structure

- setup instructions

- how to run frontend

- how to run backend

- how to seed dummy data

- future enhancements section for AI/ML matching



Keep it clean, professional, and suitable for a university final year project repository.
```

.gitignore

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Generate a proper .gitignore file for a monorepo with React frontend and FastAPI backend.



Include:
```

```
- node_modules

- dist

- build

- .env

- __pycache__

- .venv

- venv

- .pytest_cache

- .DS_Store

- coverage

- .idea

- .vscode if needed

- generated logs

- uploads if any temporary files exist


Return only the file content.
```

Backend Core Files

backend/requirements.txt

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Generate a requirements.txt for a FastAPI backend using:

- fastapi

- uvicorn

- sqlalchemy

- psycopg2-binary

- python-dotenv

- passlib[bcrypt]

- python-jose

- pydantic

- email-validator



Also include useful packages for development if appropriate.

Return only package lines.
```

backend/app/main.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a production-style FastAPI main.py file for a final year project.



Requirements:

- initialize FastAPI app

- add CORS middleware for React frontend running on localhost:5173

- include routers for auth, users, preferences, matches, groups, and dashboard

- add a simple root endpoint

- keep code modular and clean

- use best practices and comments for a student project



Return complete code.
```

backend/app/core/config.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a config.py file for FastAPI using environment variables.


Include:

- DATABASE_URL

- SECRET_KEY

- ALGORITHM

- ACCESS_TOKEN_EXPIRE_MINUTES


Use python-dotenv or pydantic settings style.

Keep it simple, clear, and beginner-friendly.

Return complete code.
```

backend/app/core/database.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a database.py file for FastAPI + SQLAlchemy + PostgreSQL.


Requirements:

- load DATABASE_URL from config

- create engine

- create SessionLocal

- create Base using declarative_base

- create a get_db dependency

- keep it clean and reusable


Return complete code.
```

backend/app/core/security.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a security.py file for FastAPI JWT authentication.
```

```
Requirements:

- password hashing with passlib bcrypt

- verify_password function

- hash_password function

- create_access_token function using python-jose

- simple token expiry handling

- clear comments for learning purposes



Return complete code.
```

Backend Models

backend/app/models/user.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a SQLAlchemy User model for a study group matching platform.


Fields:

- id

- full_name

- email

- password_hash

- role (student/admin)

- department

- academic_year

- skill_level

- preferred_study_mode

- bio

- created_at


Also include relationships where suitable.

Use clean SQLAlchemy model style.

Return complete code.
```

backend/app/models/preference.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a SQLAlchemy Preference model for a study group matching system.


Fields:

- id

- user_id

- preferred_group_size

- communication_method
```

```
- study_style

- weekly_goal_hours

- notes



Relate it properly to User.

Return complete code.
```

backend/app/models/subject.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a SQLAlchemy Subject model and a user-subject association approach for a study group platform.



Requirements:

- subject id

- subject name

- optional difficulty level

- support linking many subjects to many users



Keep it simple enough for a final year student project.

Return complete code.
```

backend/app/models/availability.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a SQLAlchemy Availability model.



Fields:

- id

- user_id

- day_of_week

- start_time

- end_time



This is for matching students based on common time slots.
```

```
Relate it properly to User.

Return complete code.
```

backend/app/models/study_group.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create SQLAlchemy models for:

1. StudyGroup

2. GroupMember


StudyGroup fields:

- id

- name

- subject_focus

- created_by

- max_members

- description

- created_at


GroupMember fields:

- id

- group_id

- user_id

- joined_at


Add relationships properly.

Return complete code.
```

backend/app/models/match.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a SQLAlchemy Match model for storing recommended matches.


Fields:

- id
```

```
- user_id

- matched_user_id

- score

- reason

- status

- created_at


This is for storing rule-based or AI-based recommendations.

Return complete code.
```

backend/app/models/message.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a simple SQLAlchemy Message model for chat between users.


Fields:

- id

- sender_id

- receiver_id

- content

- sent_at


Relate to User properly.

Keep it simple for a mid-demo project.

Return complete code.
```

Backend Schemas

backend/app/schemas/auth.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create Pydantic schemas for authentication in FastAPI.



Include:

- RegisterRequest

- LoginRequest

- TokenResponse



Use proper validation for email and password.

Return complete code.
```

backend/app/schemas/user.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create Pydantic schemas for User.



Include:

- UserCreate

- UserResponse

- UserUpdate



Match these with the User SQLAlchemy model for a study group matching platform.

Hide password hash from response.

Return complete code.
```

backend/app/schemas/preference.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create Pydantic schemas for Preference and Availability.



Include:
```

```
- PreferenceCreate

- PreferenceResponse

- AvailabilityCreate

- AvailabilityResponse


Keep it aligned with a student study matching platform.

Return complete code.
```

backend/app/schemas/match.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create Pydantic schemas for match results.


Include:

- MatchResponse

- MatchExplanationResponse


Fields should support:

- matched user basic info

- score

- reason

- overlapping subjects

- overlapping availability summary


Return complete code.
```

backend/app/schemas/study_group.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create Pydantic schemas for study groups.


Include:

- StudyGroupCreate

- StudyGroupResponse

- GroupMemberResponse
```

```
Keep it clean and appropriate for FastAPI.

Return complete code.
```

Backend Routes

backend/app/routes/auth.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a FastAPI auth router with:

- POST /register

- POST /login


Requirements:

- use SQLAlchemy session

- hash password during registration

- validate unique email

- generate JWT token on login

- return clean error messages

- use proper schema imports


Assume User model and security helpers already exist.

Return complete code.
```

backend/app/routes/users.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a FastAPI users router with:

- GET /users/me

- PUT /users/me


Requirements:

- use JWT current user dependency

- fetch logged-in user profile

- update editable profile fields

- clean and modular code

- suitable for a final year student project
```

```
Return complete code.
```

backend/app/routes/preferences.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a FastAPI preferences router with:

- POST /preferences

- GET /preferences/me

- POST /availability

- GET /availability/me


Requirements:

- save and retrieve user preferences

- save and retrieve time availability slots

- use current logged-in user

- keep code simple and clear


Return complete code.
```

backend/app/routes/matches.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a FastAPI matches router with:

- GET /matches/me


Requirements:

- call a matching_service

- compare the logged-in user with all other users

- return top recommended matches sorted by score

- include matching reasons such as same subjects, same study style, overlapping time slots,
and similar skill level


Use rule-based logic for now.

Return complete code.
```

backend/app/routes/groups.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a FastAPI groups router with:

- POST /groups

- GET /groups

- POST /groups/{group_id}/join


Requirements:

- create study groups

- list available groups

- allow logged-in user to join a group

- basic validation for duplicate joining and group size


Return complete code.
```

backend/app/routes/dashboard.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a FastAPI dashboard router with:

- GET /dashboard/summary


Return sample dashboard data for the logged-in user:

- profile completion status

- number of matches

- number of joined groups

- next available study slot count


Keep it clean and ready for React integration.

Return complete code.
```

Backend Services and Seeding

backend/app/services/matching_service.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a rule-based matching service in Python for a study group matching platform.


Goal:

calculate compatibility score between the current user and other users.


Scoring criteria:

- same subjects

- same study style

- overlapping availability

- similar skill level

- same communication preference


Requirements:

- modular helper functions

- final score out of 100

- generate a human-readable reason for each match

- return top matches sorted descending


Use clean Python code suitable for FastAPI integration.

Return complete code.
```

backend/app/services/dummy_data_service.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a Python service that generates dummy student data for a study group matching platform.


Generate around 50 students with:

- names

- emails
```

```
- departments
- academic years
- skill levels
- study modes
- subjects
- preferences
- availability slots


Use realistic university-related values.
Keep it structured so it can later be inserted into PostgreSQL.
Return complete code.
```

backend/app/services/recommendation_service.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a simple recommendation service wrapper for future AI/ML enhancement.


For now:
- explain that rule-based matching is being used
- provide a function placeholder for future ML model prediction
- include comments showing how future model training can be connected


Keep it professional and beginner-friendly.
Return complete code.
```

backend/app/seed/seed_data.py

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a Python seed_data.py script for PostgreSQL using SQLAlchemy.


Requirements:
- create database tables if needed
- insert around 50 dummy users
- insert preferences
- insert subject associations
```

```
- insert availability records

- avoid duplicate seeding if data already exists

- organize code clearly with helper functions


This is for a final year project demo.

Return complete code.
```

Frontend Core Files

frontend/src/main.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a React main.jsx entry file for a Vite project.



Requirements:

- render App

- wrap app with BrowserRouter

- wrap app with AuthContext provider

- keep code simple and clean



Return complete code.
```

frontend/src/App.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a clean App.jsx file for a React Vite project.



Requirements:

- render AppRouter

- keep file minimal

- suitable for a modular final year project structure



Return complete code.
```

frontend/src/api/axios.js

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create an axios.js setup file for React.



Requirements:

- create axios instance

- base URL should point to FastAPI backend
```

```
- automatically attach JWT token from localStorage if available

- export the axios instance



Keep it clean and production-like.

Return complete code.
```

Frontend Auth and Router

frontend/src/context/AuthContext.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a React AuthContext for JWT authentication.


Requirements:

- store logged-in user state

- store token in localStorage

- login function

- logout function

- check auth persistence on refresh

- expose context hook


Keep it beginner-friendly but professional.

Return complete code.
```

frontend/src/router/AppRouter.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a React router configuration for this project.


Pages:

- Login

- Register

- Dashboard

- Preferences

- Matches

- Groups

- Profile


Requirements:

- public routes for login and register
```

```
- protected routes for all dashboard pages

- use a ProtectedRoute component

- clean modular structure



Return complete code.
```

Frontend Components

frontend/src/components/ProtectedRoute.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a React ProtectedRoute component.


Requirements:

- check auth token or auth context

- redirect unauthenticated users to login

- render children if authenticated


Use react-router-dom.

Return complete code.
```

frontend/src/components/Navbar.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a modern React Navbar component for a student study matching platform.


Requirements:

- app title

- logged-in user name display

- logout button

- clean professional UI using simple CSS classes or Tailwind-style class names

- suitable for dashboard layout


Return complete code.
```

frontend/src/components/Sidebar.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a React Sidebar component for dashboard navigation.


Menu items:
```

```
- Dashboard
- Profile
- Preferences
- Matches
- Groups


Requirements:
- highlight active link
- clean student project UI
- responsive-friendly structure


Return complete code.
```

frontend/src/components/MatchCard.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a React MatchCard component.


Display:
- matched student name
- department
- skill level
- score
- reason for match
- communication preference if available


Make it professional and easy to read.
Return complete code.
```

frontend/src/components/GroupCard.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a React GroupCard component.


Display:
```

```
- group name

- subject focus

- max members

- current member count

- description

- join button


Keep the layout clean and suitable for a demo.

Return complete code.
```

Frontend Pages

frontend/src/pages/Login.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a professional React Login page.


Requirements:

- email and password fields

- submit to backend login API

- save token using AuthContext

- navigate to dashboard after login

- display validation and error messages

- clean UI suitable for final year demo


Return complete code.
```

frontend/src/pages/Register.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a professional React Register page.


Fields:

- full name

- email

- password

- department

- academic year

- skill level

- preferred study mode


Requirements:

- submit to backend register API

- clean UI
```

```
- validation

- redirect to login or dashboard after success



Return complete code.
```

frontend/src/pages/Dashboard.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a React Dashboard page for a study group matching platform.



Sections:

- welcome banner

- profile completion summary

- quick stats

- top recommended matches preview

- joined groups preview



Fetch summary from backend API.

Use clean modern layout.

Return complete code.
```

frontend/src/pages/Profile.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a React Profile page.



Requirements:

- fetch current user profile

- allow editing of profile fields

- save updates to backend

- clean layout

- suitable for student final year project demo



Return complete code.
```

frontend/src/pages/Preferences.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a React Preferences page.


Requirements:
- form for study style
- communication method
- group size preference
- weekly study goal
- availability slots
- submit preferences to backend
- show saved values if already available


Keep it user-friendly and clean.
Return complete code.
```

frontend/src/pages/Matches.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a React Matches page.


Requirements:
- fetch match recommendations from backend
- display them using MatchCard component
- show score and reasons clearly
- add loading and empty states
- clean demo-ready UI


Return complete code.
```

frontend/src/pages/Groups.jsx

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Create a React Groups page.
```

```
Requirements:
- show available study groups
- allow creating a new group
- allow joining a group
- use GroupCard component
- include loading states and success messages


Return complete code.
```

Extra Helper Prompts

Dummy dataset design prompt

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Generate a realistic dummy dataset design for a university study group matching platform.



Need around 50 student records with:

- full name

- email

- department

- academic year

- skill level

- preferred study mode

- selected subjects

- study style

- communication preference

- weekly study goal

- availability time slots



Also generate:

- 10 study groups

- 30+ match records with score and reason



Give it in structured JSON-style format so it can be converted into PostgreSQL seed data.

Keep values realistic for a Sri Lankan university context.
```

ML explanation prompt

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
Write a simple professional explanation for how AI/ML can be added later to a study group
matching platform.



Explain:

- current system uses rule-based matching
```

```
- future system can collect interaction data

- possible ML features: successful matches, user engagement, accepted recommendations

- model could predict better recommendations over time

- why rule-based is the best first version for a mid demo



Keep it suitable for Chapter 5 or demo presentation notes.
```

Master prompt for any single file

Copy this prompt exactly into your AI coding assistant and save the generated result in the same file path.

```
I am building a final year project called AI-Powered Adaptive Study Group Matching Platform
using React, FastAPI, PostgreSQL, SQLAlchemy, and JWT.



I want to build it in a proper professional structure for a mid demo.



Rules:

- keep the code beginner-friendly but production-style

- do not overcomplicate

- use modular folders

- use rule-based matching first

- dummy data should support 50 students

- each file should be complete and runnable

- include comments only where useful

- keep naming clean and consistent



Now generate the file: [PASTE FILE PATH HERE]



Also explain:

1. what this file does

2. where to place it

3. how it connects with other files

4. anything I must create next
```

What to build first today

- Create the backend folder and generate requirements.txt, config.py, database.py, security.py, user.py, and main.py first.
- Run the backend and fix import errors before writing every other file.
- Generate auth.py, register and login, then test with Postman or FastAPI docs.
- Add dummy data and seed the database so your UI has realistic records to show.
- Build Login, Register, Dashboard, Preferences, Matches, and Groups pages in that order.
- Keep the match engine rule-based for now and present ML as a future enhancement in the demo.